ESOFT METRO CAMPUS • FACULTY OF COMPUTING

BIT (Hons) in Software Engineering Degree Programme

DUTCH POINT NEGOMBO • HOTEL MANAGEMENT SYSTEM

System Architecture & End-to-End Execution Flow Technical Report

> **KEY ARCHITECTURAL HIGHLIGHT: DECOUPLED THREE-TIER ENTERPRISE ARCHITECTURE**
> This document provides the exhaustive technical specification of the system architecture for the Dutch Point Negombo Web-Based Hotel Management System. It details the structural responsibilities of Tier 1 (React 18 SPA), Tier 2 (ASP.NET Core 8 Web API), and Tier 3 (MySQL 8.0), followed by an under-the-hood walkthrough of the complete request-response cycle, concurrency locking, and security controls.

Project Metadata & Presenting Engineering Team

Student Name	Student ID	Project Architectural Role	Component Responsibility
I.D.P.N Iddamalgoda	E269559	Lead Developer & System Admin	Three-Tier Integration, Security & Concurrency Verification
H.M.Y.R Herath	E267614	Backend Software Engineer	ASP.NET Core 8 Web API, C# Controllers & EF Core ORM Layer
Y.M.A.Y Bandara	E267609	Frontend Software Engineer	React 18 SPA Client, State Management & Asynchronous Fetch API
A.G.H.H Jinadasa	E267612	QA & Requirements Engineer	Architectural Testing, Load Validation & Requirements Traceability

1. Architectural Overview & Design Pattern

The Dutch Point Negombo Hotel Management System employs an enterprise-grade Three-Tier Decoupled Architecture. Unlike monolithic web applications where HTML generation, business calculations, and database queries are combined inside a single process, this system strictly separates presentation concerns, business rules, and persistent data storage into three distinct physical and logical layers:

Architectural Layer	Underlying Technology	Execution Environment & Port	Core Architectural Purpose
Tier 1: Presentation Tier	React 18, Vite, Bootstrap 5, CSS3	Client Web Browser (Port 5173 / Localhost)	User experience, responsive rendering, form state management & JSON serialization.

Tier 2: Application Tier	ASP.NET Core 8 Web API, C#, EF Core 8	Host Server Runtime (Port 5000 / HTTP)	RESTful routing, RBAC authorization, business rules, concurrency checks & ORM compilation.
Tier 3: Data Tier	MySQL Server 8.0, InnoDB Storage Engine	Database Daemon (Port 3306 / TCP)	ACID-compliant storage, foreign key constraints, table indexing & transaction journaling.

The decoupling between Tier 1 and Tier 2 is achieved through standardized, asynchronous RESTful APIs communicating over HTTP. The frontend has zero direct connection to the database, ensuring that client-side scripts can never execute arbitrary SQL or bypass business validation rules.

2. Comprehensive Breakdown of the Three Architectural Tiers

2.1 Tier 1: Presentation Tier (Frontend Client)

The Presentation Tier is executed entirely on the client's device inside a modern web browser. Key architectural components include:

- **Component-Based Single Page Application (SPA):** Constructed using React 18 and compiled with Vite. Navigation between views (Home, Luxury Rooms, Occupancy Tracker, My Bookings, Admin Dashboard) occurs without reloading the web page, ensuring near-instantaneous transitions.
- **Client-Side State Management:** Uses React Hooks (useState, useEffect, useMemo) to maintain real-time filter states (Check-In Date, Check-Out Date, Guest Headcount) and user session tokens in memory.
- **Responsive Layout & Boutique Styling:** Utilizes Bootstrap 5 grid systems combined with bespoke CSS variables to deliver an executive coastal boutique aesthetic tailored for Dutch Point Negombo.
- **Asynchronous REST Communications:** Fires asynchronous browser fetch() requests to the backend API, passing structured JSON payloads and dynamically rendering returned data into DOM elements.

2.2 Tier 2: Application / Business Logic Tier (Backend API)

The Application Tier is built using C# and ASP.NET Core 8 Web API. It functions as the secure gatekeeper and brain of the system:

- **Domain-Driven REST Controllers:** Incoming requests are categorized into specialized controllers: AuthController (JWT tokens and registration), RoomsController (inventory and status), BookingsController (transactional reservation lifecycle), and DashboardController (operational KPIs).
- **Model Binding & Data Transfer Objects (DTOs):** Incoming JSON strings are automatically deserialized into strongly-typed C# DTOs (e.g., CreateBookingRequest, OccupiedRoomDto), validating data types and rejecting malformed inputs before reaching business logic.
- **Business Rule & Pricing Enforcement:** Computes stay totals strictly in Sri Lankan Rupees (LKR) using the formula: TotalAmount = Nights * PricePerNight. Validates headcount against room limits (Capacity >= NumberOfGuests).
- **Transactional Concurrency Engine:** Prevents overbooking by verifying that requested stay intervals do not overlap with existing confirmed reservations inside an ACID database transaction.
- **Entity Framework Core (EF Core 8) ORM:** Translates high-level LINQ queries into optimized, parameterized SQL statements executed against the MySQL database.

2.3 Tier 3: Data Tier (Relational Database Management System)

The Data Tier is managed by MySQL Server 8.0 running the normalized relational schema 'hotel_db':

- **Normalized Relational Schema (3NF):** Organized into four primary business entities: customers (user identity and roles), rooms (physical suites, LKR rates, capacities), bookings (reservation dates, headcounts, statuses), and payments (financial vouchers).
- **Referential Integrity Constraints:** Foreign keys strictly enforce parent-child relationships between Customers, Rooms, and Bookings. ON DELETE RESTRICT prevents accidental deletion of rooms or guest records tied to active financial reservations.
- **Financial Precision:** All monetary values are stored in DECIMAL(10,2) columns, preventing floating-point rounding errors during currency calculations in LKR.
- **ACID Transaction Guarantees:** Atomicity, Consistency, Isolation, and Durability ensure that reservations and payment records are written completely or rolled back entirely upon server failure.

3. 'How It Happens': The Complete End-to-End Execution Flow

To understand how the three tiers collaborate during real-world operations, consider the complete execution lifecycle when a resort guest books Deluxe Suite 101 for October 10th to October 12th, 2026:

Phase & Step	Executing Tier & File	Action Performed Under the Hood	Data Format / Protocol
Step 1: User Input	Tier 1: React Client (App.jsx)	Guest selects check-in (2026-10-10), check-out (2026-10-12), 2 guests, and clicks 'Confirm Booking'.	In-Memory React State
Step 2: Request Packaging	Tier 1: BookingModal.jsx	Client serializes inputs into JSON: { customerId: 1, roomId: 3, checkInDate: '2026-10-10', ... } and initiates fetch().	HTTP POST Payload (JSON)
Step 3: Network Transport	Network Layer (Port 5000)	Request travels over TCP to ASP.NET Core listening on http://localhost:5000/api/bookings.	TCP / IP Protocol
Step 4: Middleware Routing	Tier 2: Program.cs Pipeline	CORS middleware inspects origin. Routing middleware maps URL to BookingsController.CreateBooking().	ASP.NET Core Middleware
Step 5: Model Binding	Tier 2: BookingsController.cs	ASP.NET Core deserializes JSON into C# CreateBookingRequest object. Validates checkIn < checkOut.	Strongly-Typed C# Object
Step 6: Capacity Verification	Tier 2: BookingsController.cs	Backend checks room capacity: Does Suite 101 hold 2 guests? (Capacity 2 >= 2 -> Approved).	C# In-Memory Evaluation
Step 7: Concurrency Query	Tier 2: EF Core 8 (LINQ)	Executes overlap LINQ check: Any existing booking where CheckIn < ReqOut AND CheckOut > ReqIn?	LINQ to Entities Expression
Step 8: SQL Parameterization	Tier 2: EF Core Provider	Translates LINQ to parameterized SQL: SELECT EXISTS(...) WHERE RoomId = @p0 AND ...	Parameterized SQL Statement
Step 9: Database Execution	Tier 3: MySQL (hotel_db)	MySQL engine executes query on Port 3306. Evaluates index. Returns false (Room is currently vacant).	Binary Database Result Set
Step 10: Financial Calculation	Tier 2: BookingsController.cs	Calculates stay: (Oct 12 - Oct 10) = 2 nights. Total = 2 * LKR 65,000 = LKR 130,000.	C# Decimal Arithmetic
Step 11: Transactional Insert	Tier 3: MySQL Engine	EF Core dispatches INSERT INTO bookings... MySQL writes record to disk and issues auto-increment ID 7.	ACID SQL Transaction Commit
Step 12: Success	Tier 2: BookingsController.cs	Serializes success payload: { message:	HTTP 200 OK (JSON)

Response		'Booking created!', bookingId: 7, totalAmount: 130000.00 } and returns HTTP 200.	
Step 13: UI Re-rendering	Tier 1: App.jsx	fetch() receives 200 OK. React closes modal, displays confirmation banner, and triggers live occupancy refresh.	DOM Re-render (Virtual DOM)

3.1 Branching Logic: What Happens When a Collision Occurs?

If a second guest attempts to reserve the identical suite for overlapping dates (e.g., October 11th to 14th), Step 9 detects the existing record committed in Step 11. The backend immediately aborts execution, generates an HTTP 409 Conflict status, and returns the existing booking information. The React frontend intercepts the 409 status and renders a red alert banner, completely preventing the second reservation from reaching the database.

```
// Server-Side Concurrency Guard (BookingsController.cs)
var conflictingBooking = await _context.Bookings
    .Where(b => b.RoomId == request.RoomId &&
               (b.BookingStatus == "Confirmed" || b.BookingStatus == "Checked-In") &&
                b.CheckInDate < checkOut &&
                b.CheckOutDate > checkIn)
    .FirstOrDefaultAsync();

if (conflictingBooking != null)
{
    return Conflict(new { message = "Room is already booked for the selected dates." });
}
```

4. Architectural Security & Integrity Controls

The three-tier architecture incorporates multi-layered defense mechanisms:

- **Zero Client Trust Principle:** The browser client is treated as inherently untrusted. Critical calculations (room prices, stay durations, role permissions) are performed exclusively on the Application Tier. Room rates are queried directly from MySQL, preventing client-side price tampering.
- **Server-Side Role Isolation:** To eliminate OWASP Broken Access Control vulnerabilities, public registration endpoints ignore client-supplied role parameters and strictly hardcode the 'Customer' role on the server. Administrative roles can only be granted directly in the database.
- **SQL Injection Neutralization:** All database interactions occur via Entity Framework Core LINQ queries, which compile into strongly-parameterized SQL commands (@p0, @p1). User input is treated strictly as literal data, rendering SQL injection exploits impossible.
- **Cross-Origin Resource Sharing (CORS):** The ASP.NET Core pipeline explicitly whitelists the frontend development origin (http://localhost:5173), rejecting unauthorized cross-site API invocations.

5. Architectural Advantages & Viva Defense Justification

When defending the architectural design to the examination panel, emphasize these four core software engineering benefits:

Architectural Benefit	Software Engineering Principle	Practical Value to Dutch Point Negombo
Separation of Concerns	Modularity & Decoupling	Frontend engineers can redesign the boutique UI without touching database schemas; backend engineers can optimize SQL queries without breaking the UI.
Platform Extensibility	Reusability of Business Logic	The same ASP.NET Core REST API can serve a native mobile app (iOS/Android) or front-desk desktop tablet in the future with zero backend changes.
Maintainability & Isolation	Single Responsibility Principle	Authentication issues in AuthController are isolated from reservation logic in BookingsController, minimizing software regression risks.
Optimal Resource Usage	Client-Side Processing	The browser handles DOM rendering, animations, and form validation, relieving the server from rendering HTML templates and enabling higher guest throughput.